

Prompt Log — Sight Word Match (Word Match)

Game: Word Match

Scoring Model: (matches / attempts) × 100, rounded to 0–100

#	Prompt to AI	AI Response Summary	Outcome / Notes
1	I'm starting a Flutter project for a school assignment. It's a sight word matching game for kids, and it's like the card concentration game. I have the Dolch word lists and need to organize them by level. Can you help me think through what files I'll need and how to set up the project structure before I start coding?	Suggested a folder layout under lib/: - data/dolch_words.dart - word lists by level - models/game_result.dart - stores one round's result - screens/ - home, game, results, stats screens - services/score_storage_service.dart - handles saving/loading scores Explained the basic Navigator.push() pattern for moving between screens.	I used this layout as a blueprint for the whole project. Having a clear structure upfront made it easy to know where each piece of code should go.
2	I'm working on the home screen now. I want it to show the five Dolch levels as big colorful cards that kids can tap to pick a level. I already know I want a purple app bar with a stats button. I'm not sure how to lay out the cards or how to make each one a different color, can you point me in the right direction?	Recommended a ListView.separated of reusable card widgets. Showed how to use Material + InkWell with BorderRadius for tappable rounded cards. Suggested storing the five level colors in a static list and indexing into it. Demonstrated Navigator.push() to pass the chosen level into the game screen.	I implemented the home screen using these suggestions. The color-coded cards matched the child-friendly design goal and the code stayed clean with a small private widget for each card.
3	I'm trying to figure out how the game board should work. I want to pick 6 random words from the chosen level, make two cards for each word, shuffle the whole deck, and show them in a	Suggested a small _WordCardData class with isFaceUp and isMatched booleans.	The game logic worked correctly on the first run. The _isChecked flag prevented a bug where rapid taps could flip a third card during the delay window.

#	Prompt to AI	AI Response Summary	Outcome / Notes
	3-column grid. When two face-up cards match they stay visible, and if they don't, they flip back after a short pause. I'm a little unsure about how to track which cards are flipped and how to prevent the user from tapping a third card during the delay. Any guidance?	Recommended a <code>_flippedIndices</code> list capped at 2 items to track the active pair. Explained using a bool <code>_isChecking</code> flag during the flip-back delay to block extra taps. Showed <code>Future.delayed</code> for the 700ms pause before flipping unmatched cards back. Confirmed <code>GridView.builder</code> with <code>crossAxisCount: 3</code> for the 3-column layout.	
4	The assignment says scores have to persist, closing and reopening the app shouldn't lose them. I've heard of <code>SharedPreferences</code> for Flutter. I want to save each completed round with the game type, level, score (0–100), and the date played. I also need to compute a combined score across all rounds. Can you help me understand how to structure that?	Explained using <code>shared_preferences</code> to store a JSON-encoded list under one key. Suggested a <code>GameResult</code> class with <code>toJson()</code> and <code>fromJson()</code> for easy serialization. Walked through a <code>saveResult()</code> method that loads the existing list, appends, and re-saves. Showed how <code>getCombinedScore()</code> can return null when no games have been played yet, which makes empty-state handling straightforward.	Scores persisted correctly across app restarts. Returning null from <code>getCombinedScore()</code> when the list is empty cleanly handled the 'no games yet' case without extra flags.
5	I'm running into two issues. First, if I tap 'Play Again' and 'Home' really fast on the results screen, it seems like both buttons fire at once and something weird happens with the navigation stack. Second, my stats screen crashes when I open it before playing any games. How do I fix these?	Double-tap fix: suggested an <code>_actionTaken</code> boolean, and set it to true on the first button press and return early on any subsequent tap before the navigation completes. Stats empty-state fix: check if the results list is empty inside the <code>FutureBuilder</code> and render a friendly	Both bugs resolved. The double-tap guard stopped the navigation stack from getting into a bad state, and the stats screen now shows a clear message when no games have been played yet.

#	Prompt to AI	AI Response Summary	Outcome / Notes
		message instead of trying to compute averages on an empty list. Confirmed this satisfies the assignment's requirement for graceful empty-state handling.	
6	For the README I need to explain and defend my scoring model. My score formula is: $(\text{matches} / \text{attempts}) * 100$. The professor wants me to explain why I chose this approach, what the tradeoff is, and when it might stop working. Can you help me think through how to write that clearly?	Explained the reasoning: an accuracy ratio rewards careful play over rushed guessing, which fits a young child audience where speed pressure isn't the goal. Identified the main tradeoff: a lucky player who guesses correctly early will score higher than a methodical player who makes a few extra attempts, and luck isn't filtered out. Noted the boundary condition: this model works well for a single matching game, but averaging scores across fundamentally different game types would eventually lose meaning. Pointed out that storing gameType separately in GameResult means scoring can be adjusted later without rewriting the storage layer.	I used these points to write the README defense. Having concrete reasoning for each part of the tradeoff made the argument specific rather than vague.